

Teori PyTorch untuk Deep Learning

Ryan Fadhilah Faizal Hakim¹

¹Sains Data, Fakultas Sains dan Teknologi, IKOPIN University

1 Pendahuluan

PyTorch adalah pustaka komputasi numerik dan deep learning berbasis tensor yang banyak digunakan untuk riset, prototipe, dan produksi model kecerdasan buatan. PyTorch menyediakan struktur data utama berupa *tensor*, sistem diferensiasi otomatis bernama *autograd*, modul pembangunan jaringan saraf melalui `torch.nn`, optimisasi melalui `torch.optim`, pemrosesan data melalui `torch.utils.data`, serta dukungan akselerasi perangkat keras seperti GPU CUDA, Apple MPS, dan backend komputasi lain.

Dalam deep learning, PyTorch berperan sebagai kerangka kerja untuk mendefinisikan fungsi model f_θ , menghitung loss $\mathcal{L}$, melakukan propagasi balik untuk memperoleh gradien $\nabla_\theta \mathcal{L}$, dan memperbarui parameter menggunakan algoritma optimisasi. Kekuatan utama PyTorch terletak pada *dynamic computation graph*, yaitu graf komputasi dibangun secara dinamis saat kode Python dieksekusi. Hal ini membuat PyTorch fleksibel untuk model kompleks seperti recurrent neural network, transformer, model generatif, graph neural network, dan arsitektur riset yang sering berubah.

Secara konseptual, pelatihan model deep learning di PyTorch mengikuti masalah optimisasi berikut:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(\mathbf{x}^{(i)}), y^{(i)}), \quad (1)$$

dengan θ adalah parameter model, $\mathbf{x}^{(i)}$ adalah input, $y^{(i)}$ adalah target, f_θ adalah jaringan saraf, dan $\mathcal{L}$ adalah fungsi loss.

2 Tensor sebagai Fondasi PyTorch

Tensor adalah generalisasi skalar, vektor, matriks, dan array multidimensi. Dalam PyTorch, tensor direpresentasikan oleh `torch.Tensor`. Jika skalar memiliki orde 0, vektor orde 1, dan matriks orde 2, maka tensor dapat memiliki orde n .

Contoh bentuk tensor:

- Skalar: $x \in \mathbb{R}$, bentuk `()`
- Vektor: $\mathbf{x} \in \mathbb{R}^d$, bentuk `(d,)`
- Matriks: $\mathbf{X} \in \mathbb{R}^{m \times n}$, bentuk `(m, n)`
- Batch citra: $\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W}$, bentuk `(batch, channel, height, width)`
- Batch token teks: $\mathbf{X} \in \mathbb{N}^{B \times T}$, bentuk `(batch, sequence length)`

Kode dasar tensor:

```
import torch

x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
y = torch.zeros(2, 3)
z = torch.randn(4, 5)
print(x.shape, x.dtype, x.device)
```

Setiap tensor memiliki atribut penting:

- `shape`: ukuran setiap dimensi.
- `dtype`: tipe data, misalnya `torch.float32`, `torch.float16`, `torch.int64`.
- `device`: lokasi tensor, misalnya CPU atau GPU.
- `requires_grad`: apakah autograd perlu melacak operasi pada tensor.
- `grad`: tempat penyimpanan gradien setelah `backward()`.

3 Operasi Tensor dan Broadcasting

PyTorch menyediakan operasi aritmetika, aljabar linear, reduksi, indexing, slicing, dan transformasi bentuk. Operasi tensor biasanya bersifat vektor sehingga jauh lebih efisien daripada loop Python.

```
a = torch.randn(3, 4)
b = torch.randn(3, 4)

c = a + b
m = a @ b.T
s = a.sum(dim=1)
r = a.reshape(2, 6)
```

Broadcasting memungkinkan operasi antara tensor dengan bentuk berbeda selama dimensi-dimensinya kompatibel. Misalnya, tensor berukuran (B, d) dapat dijumlahkan dengan tensor berukuran $(d,)$ karena vektor $(d,)$ diperluas secara logis sepanjang dimensi batch.

Jika $\mathbf{X} \in \mathbb{R}^{B \times d}$ dan $\mathbf{b} \in \mathbb{R}^d$, maka operasi

$$\mathbf{Y}_{ij} = \mathbf{X}_{ij} + \mathbf{b}_j \quad (2)$$

dapat dilakukan langsung di PyTorch:

```
X = torch.randn(32, 10)
bias = torch.randn(10)
Y = X + bias
```

Broadcasting penting untuk bias layer, normalisasi, attention mask, dan manipulasi batch.

4 Device: CPU, GPU, CUDA, dan MPS

PyTorch dapat menjalankan tensor dan model pada CPU atau akselerator. Pada komputer dengan NVIDIA GPU, backend yang umum digunakan adalah CUDA. Pada perangkat Apple Silicon, backend yang dapat digunakan adalah MPS.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
inputs = inputs.to(device)
targets = targets.to(device)
```

Aturan penting: semua tensor yang terlibat dalam operasi yang sama harus berada pada device yang sama. Kesalahan umum adalah model berada di GPU, tetapi data masih berada di CPU.

5 Autograd dan Diferensiasi Otomatis

Autograd adalah sistem diferensiasi otomatis PyTorch. Ketika tensor diatur untuk membutuhkan gradien, PyTorch melacak operasi pada tensor tersebut dan membangun graf komputasi dinamis. Setelah nilai loss dihitung, pemanggilan `loss.backward()` menghitung gradien terhadap semua tensor leaf yang membutuhkan gradien.

Misalkan fungsi sederhana:

$$y = x^2 + 3x + 1. \quad (3)$$

Turunannya adalah:

$$\frac{dy}{dx} = 2x + 3. \quad (4)$$

Di PyTorch:

```
x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x + 1
y.backward()
print(x.grad) # tensor(7.)
```

Graf komputasi terdiri dari node operasi dan edge yang membawa tensor. PyTorch menggunakan *reverse-mode automatic differentiation*, yang efisien untuk deep learning karena jumlah parameter sangat besar dan loss biasanya skalar.

6 Computation Graph Dinamis

Berbeda dari graf statis, graf PyTorch dibangun ulang setiap forward pass. Artinya percabangan, loop, dan struktur kontrol Python dapat mempengaruhi arsitektur komputasi.

```
def dynamic_forward(x):  
    if x.mean() > 0:  
        return x ** 2  
    return torch.abs(x)
```

Fleksibilitas ini berguna untuk model dengan panjang sekuens berubah, arsitektur adaptif, reinforcement learning, dan riset model baru.

7 Gradien, Akumulasi Gradien, dan Zero Grad

Secara default, PyTorch mengakumulasi gradien. Jika `backward()` dipanggil beberapa kali tanpa menghapus gradien, gradien baru akan ditambahkan ke gradien lama. Oleh karena itu, dalam training loop hampir selalu digunakan:

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Jika parameter θ memiliki gradien g_t , optimizer melakukan pembaruan seperti:

$$\theta_{t+1} = \theta_t - \eta g_t, \quad (5)$$

dengan η adalah learning rate.

8 Modul torch.nn

`torch.nn` menyediakan komponen untuk membangun jaringan saraf. Kelas utama adalah `nn.Module`. Semua model PyTorch sebaiknya mewarisi kelas ini.

```
import torch.nn as nn  
  
class MLP(nn.Module):  
    def __init__(self, input_dim, hidden_dim, output_dim):  
        super().__init__()
```

```

        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        return self.net(x)

```

Komponen penting `nn.Module`:

- `__init__`: mendefinisikan layer dan parameter.
- `forward`: mendefinisikan alur komputasi.
- `parameters()`: mengembalikan parameter yang dapat dilatih.
- `state_dict()`: menyimpan parameter dan buffer model.
- `train()` dan `eval()`: mengatur mode training atau evaluasi.

9 Parameter, Buffer, dan State Dict

Parameter adalah tensor yang dipelajari optimizer. Dalam PyTorch, parameter biasanya berupa `nn.Parameter`. Layer seperti `nn.Linear` otomatis membuat parameter bobot dan bias.

Buffer adalah tensor yang bukan parameter tetapi tetap menjadi bagian dari status model, misalnya *running mean* dan *running variance* pada batch normalization.

```

for name, param in model.named_parameters():
    print(name, param.shape)

for name, buffer in model.named_buffers():
    print(name, buffer.shape)

```

`state_dict` adalah dictionary berisi parameter dan buffer. Objek ini digunakan untuk menyimpan dan memuat model.

10 Layer Dasar dalam PyTorch

10.1 Linear Layer

Layer linear melakukan transformasi affine:

$$\mathbf{y} = \mathbf{x}\mathbf{W}^T + \mathbf{b}. \quad (6)$$

Jika input memiliki bentuk (B, d_{in}) , bobot memiliki bentuk (d_{out}, d_{in}) , dan bias memiliki bentuk (d_{out}) , maka output memiliki bentuk (B, d_{out}) .

```
layer = nn.Linear(10, 5)
y = layer(torch.randn(32, 10))
```

10.2 Activation Function

Fungsi aktivasi memberi non-linearitas. Tanpa aktivasi non-linear, tumpukan layer linear tetap ekuivalen dengan satu transformasi linear.

Beberapa aktivasi umum:

$$\text{ReLU}(x) = \max(0, x), \quad (7)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (8)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad (9)$$

$$\text{GELU}(x) \approx x\Phi(x). \quad (10)$$

10.3 Convolution Layer

`nn.Conv2d` digunakan untuk citra. Untuk input $\mathbf{X} \in \mathbb{R}^{B \times C_{in} \times H \times W}$, convolution menghasilkan fitur lokal menggunakan kernel.

```
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
y = conv(torch.randn(8, 3, 64, 64))
```

10.4 Normalization Layer

Normalisasi menstabilkan pelatihan. Batch normalization menggunakan statistik batch:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta. \quad (11)$$

Layer normalization menormalisasi sepanjang dimensi fitur dan umum digunakan pada transformer.

10.5 Dropout

Dropout menonaktifkan unit secara acak saat training:

$$\tilde{h} = \frac{m \odot h}{1 - p}, \quad m_j \sim \text{Bernoulli}(1 - p). \quad (12)$$

Saat evaluasi, dropout dimatikan. Karena itu, mode `model.train()` dan `model.eval()` sangat penting.

11 Loss Function

Loss function mengukur kesalahan prediksi. PyTorch menyediakan banyak loss pada `torch.nn`.

Tugas	Loss	Catatan
Regresi	<code>nn.MSELoss</code>	Error kuadrat
Regresi robust	<code>nn.SmoothL1Loss</code>	Kurang sensitif terhadap outlier
Klasifikasi multi-kelas	<code>nn.CrossEntropyLoss</code>	Input berupa logits
Klasifikasi multi-label	<code>nn.BCEWithLogitsLoss</code>	Setiap kelas independen
Segmentasi	Cross entropy / Dice loss	Per piksel
Metric learning	Triplet / contrastive loss	Belajar embedding

Untuk klasifikasi multi-kelas, `nn.CrossEntropyLoss` menggabungkan `log_softmax` dan negative log-likelihood. Jika logits $\mathbf{z}$ dan kelas benar y , maka:

$$\mathcal{L}(\mathbf{z}, y) = -\log \frac{e^{z_y}}{\sum_k e^{z_k}}. \quad (13)$$

12 Optimizer

Optimizer memperbarui parameter berdasarkan gradien.

12.1 Stochastic Gradient Descent

SGD memperbarui parameter:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t). \quad (14)$$

Dengan momentum:

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t + \nabla_{\theta} \mathcal{L}(\theta_t), \quad (15)$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1}. \quad (16)$$

12.2 Adam

Adam menggunakan estimasi momen pertama dan kedua:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (17)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (18)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (19)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (20)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (21)$$

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

AdamW memisahkan weight decay dari gradien adaptif sehingga sering lebih stabil untuk model modern seperti transformer.

13 Dataset dan DataLoader

`Dataset` mendefinisikan cara mengambil satu sampel, sedangkan `DataLoader` mengatur batching, shuffling, parallel loading, dan collation.

```
from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

loader = DataLoader(MyDataset(X, y), batch_size=32, shuffle=True)
```

Untuk data dengan ukuran sekuens berbeda, fungsi `collate_fn` sering digunakan untuk padding atau packing.

14 Training Loop Lengkap

Training loop PyTorch eksplisit sehingga peneliti dapat mengontrol setiap tahap.


```

model = MLP(input_dim=20, hidden_dim=64, output_dim=3).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0

    for inputs, targets in train_loader:
        inputs = inputs.to(device)
        targets = targets.to(device)

        logits = model(inputs)
        loss = criterion(logits, targets)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * inputs.size(0)

    train_loss = total_loss / len(train_loader.dataset)
    print(epoch, train_loss)

```

Urutan penting training loop:

1. Set model ke mode training.
2. Pindahkan batch ke device.
3. Lakukan forward pass.
4. Hitung loss.
5. Hapus gradien lama.
6. Lakukan backward pass.
7. Perbarui parameter.
8. Catat metrik.

15 Evaluation Loop dan torch.no_grad

Saat evaluasi, gradien tidak diperlukan. `torch.no_grad()` menghemat memori dan komputasi.

```

model.eval()
correct = 0
count = 0

```

```

with torch.no_grad():
    for inputs, targets in val_loader:
        inputs = inputs.to(device)
        targets = targets.to(device)
        logits = model(inputs)
        predictions = logits.argmax(dim=1)
        correct += (predictions == targets).sum().item()
        count += targets.numel()

accuracy = correct / count

```

Mode evaluasi penting karena layer seperti dropout dan batch normalization memiliki perilaku berbeda antara training dan evaluasi.

16 Shape Management

Banyak error PyTorch berasal dari bentuk tensor yang tidak sesuai. Prinsip umum:

- Linear layer menerima fitur terakhir berukuran `in_features`.
- Conv2d biasanya memakai format (B, C, H, W).
- CrossEntropyLoss untuk klasifikasi menerima logits (B, K) dan target (B) bertipe `long`.
- Transformer sering memakai format (B, T, d) atau (T, B, d), tergantung konfigurasi.

Operasi penting untuk shape:

```

x = x.view(batch_size, -1) # reshape dengan memori contiguous
x = x.reshape(batch_size, -1) # lebih fleksibel
x = x.permute(0, 2, 3, 1) # ubah urutan dimensi
x = x.unsqueeze(1) # tambah dimensi
x = x.squeeze(1) # hapus dimensi berukuran 1

```

17 In-Place Operation dan Autograd

Operasi in-place mengubah tensor langsung, misalnya `x.add_(1)` atau `relu_(x)`. Operasi ini dapat menghemat memori, tetapi berisiko mengganggu autograd jika tensor lama masih dibutuhkan untuk backward pass. Jika muncul error tentang tensor yang telah dimodifikasi in-place, gunakan operasi out-of-place.

18 Regularisasi dalam PyTorch

Regularisasi mencegah overfitting. Teknik umum:

- Weight decay melalui optimizer.
- Dropout melalui `nn.Dropout`.
- Data augmentation pada pipeline dataset.
- Early stopping berdasarkan loss validasi.
- Label smoothing pada `nn.CrossEntropyLoss(label_smoothing=...)`.
- Gradient clipping untuk RNN dan transformer.

Contoh gradient clipping:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

19 Learning Rate Scheduler

Learning rate scheduler mengubah learning rate selama training. Strategi umum meliputi step decay, cosine annealing, warmup, dan reduce-on-plateau.

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)

for epoch in range(100):
    train_one_epoch()
    scheduler.step()
```

Pada transformer, warmup sering digunakan:

$$\eta_t = d_{model}^{-1/2} \min(t^{-1/2}, t \cdot warmup^{-3/2}). \quad (22)$$

20 Mixed Precision Training

Mixed precision menggunakan kombinasi `float32` dan `float16/bfloat16` untuk mempercepat training dan menghemat memori. PyTorch menyediakan automatic mixed precision melalui `torch.autocast` dan gradient scaling.

```

scaler = torch.cuda.amp.GradScaler()

for inputs, targets in train_loader:
    optimizer.zero_grad()

    with torch.cuda.amp.autocast():
        logits = model(inputs)
        loss = criterion(logits, targets)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

```

Gradient scaling diperlukan karena `float16` dapat mengalami underflow pada gradien kecil.

21 Saving dan Loading Model

Cara yang direkomendasikan adalah menyimpan `state_dict`, bukan seluruh objek model.

```

torch.save(model.state_dict(), "model.pt")

model = MLP(input_dim=20, hidden_dim=64, output_dim=3)
model.load_state_dict(torch.load("model.pt", map_location="cpu"))
model.eval()

```

Untuk melanjutkan training, simpan juga optimizer, scheduler, epoch, dan metrik.

```

checkpoint = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "epoch": epoch,
    "best_val_loss": best_val_loss,
}
torch.save(checkpoint, "checkpoint.pt")

```

22 Transfer Learning

Transfer learning memanfaatkan model pra-latih. Dalam PyTorch, pendekatan umum adalah membekukan sebagian layer dan mengganti head klasifikasi.

```

for param in backbone.parameters():
    param.requires_grad = False

model.classifier = nn.Linear(in_features, num_classes)

```

Strategi umum:

- Feature extraction: backbone dibekukan, hanya head dilatih.
- Fine-tuning parsial: beberapa blok akhir dilatih.
- Full fine-tuning: semua parameter dilatih dengan learning rate kecil.

23 PyTorch untuk CNN

Arsitektur CNN biasanya terdiri dari convolution, activation, pooling, normalization, dan fully connected head.

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1))
        )
        self.classifier = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)
```

24 PyTorch untuk RNN, LSTM, dan GRU

RNN memproses data sekuens. LSTM dan GRU mengatasi masalah vanishing gradient melalui mekanisme gate. Dalam PyTorch, input RNN dapat berbentuk (batch, seq, feature) jika `batch_first=True`.

```
lstm = nn.LSTM(input_size=128, hidden_size=256, num_layers=2, batch_first=True)
x = torch.randn(32, 50, 128)
out, (h_n, c_n) = lstm(x)
```

Output `out` berisi hidden state untuk setiap timestep, sedangkan `h_n` berisi hidden state akhir setiap layer.

25 PyTorch untuk Transformer

Transformer menggunakan self-attention. Untuk query Q , key K , dan value V , scaled dot-product attention adalah:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (23)$$

PyTorch menyediakan `nn.TransformerEncoderLayer`, `nn.MultiheadAttention`, dan fungsi `attention` yang dioptimalkan pada versi modern.

```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=256,
    nhead=8,
    dim_feedforward=1024,
    batch_first=True
)
encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)
x = torch.randn(16, 128, 256)
y = encoder(x)
```

Komponen teknis transformer:

- Token embedding mengubah indeks token menjadi vektor.
- Positional encoding menambahkan informasi urutan.
- Multi-head attention mempelajari beberapa relasi sekaligus.
- Feed-forward network memperkaya representasi tiap token.
- Residual connection dan layer normalization menstabilkan training.
- Attention mask mencegah token melihat posisi tertentu, misalnya masa depan pada language model.

26 Debugging PyTorch

Strategi debugging penting:

- Cetak shape tensor pada setiap tahap forward pass.
- Pastikan target memiliki tipe data yang sesuai.
- Periksa apakah loss menjadi NaN atau infinity.
- Gunakan batch kecil untuk overfit satu batch sebagai sanity check.

- Periksa gradien dengan `param.grad.norm()`.
- Gunakan `torch.autograd.set_detect_anomaly(True)` untuk melacak error backward.

Contoh inspeksi gradien:

```
for name, param in model.named_parameters():
    if param.grad is not None:
        print(name, param.grad.norm().item())
```

27 Reproducibility

Eksperimen deep learning dapat bervariasi karena inisialisasi acak, urutan batch, operasi GPU non-deterministik, dan augmentasi data. Untuk meningkatkan reproducibility:

```
import random
import numpy as np
import torch

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Namun determinisme penuh dapat menurunkan performa karena membatasi algoritma backend.

28 Performance Engineering

Agar training efisien:

- Gunakan operasi tensor vektorisasi, hindari loop Python yang tidak perlu.
- Gunakan `num_workers` dan `pin_memory` pada `DataLoader` untuk GPU.
- Gunakan mixed precision jika hardware mendukung.
- Minimalkan transfer CPU-GPU di tengah training loop.
- Gunakan batch size optimal sesuai memori.
- Profiling dengan `torch.profiler` jika bottleneck tidak jelas.

```
loader = DataLoader(dataset, batch_size=128, shuffle=True,
                    num_workers=4, pin_memory=True)
```

29 TorchScript, Export, dan Produksi

Untuk produksi, model sering perlu diekspor atau dioptimalkan. PyTorch memiliki beberapa jalur:

- `torch.jit.trace`: merekam operasi dari contoh input.
- `torch.jit.script`: mengompilasi subset Python yang kompatibel.
- ONNX export: interoperabilitas dengan runtime lain.
- `torch.compile`: kompilasi graf untuk optimisasi performa pada PyTorch modern.

Contoh `torch.compile`:

```
compiled_model = torch.compile(model)
```

30 Distributed Training

Untuk model dan dataset besar, training dapat didistribusikan. Strategi umum:

- Data parallelism: setiap GPU memproses batch berbeda, gradien disinkronkan.
- Model parallelism: bagian model ditempatkan pada device berbeda.
- Pipeline parallelism: layer model dibagi menjadi tahap pipeline.
- Fully Sharded Data Parallel: parameter, gradien, dan optimizer state dibagi antar GPU.

Dalam PyTorch, `DistributedDataParallel` lebih direkomendasikan daripada `DataParallel` karena lebih efisien dan skalabel.

31 Kesalahan Umum dalam PyTorch

Masalah	Penyebab dan Solusi
Device mismatch	Pastikan model, input, dan target berada pada device yang sama.
Shape mismatch	Cetak shape dan cocokkan dengan kontrak layer/loss.
Loss tidak turun	Periksa learning rate, normalisasi data, label, dan gradien.
NaN pada loss	Turunkan learning rate, gunakan gradient clipping, cek pembagian nol.
Akurasi buruk saat evaluasi	Pastikan menggunakan <code>model.eval()</code> dan preprocessing sama.
Gradien tidak ada	Periksa <code>requires_grad</code> , <code>detach()</code> , dan penggunaan <code>no_grad</code> .
Memori GPU habis	Kurangi batch size, gunakan mixed precision, gradient checkpointing.

32 Contoh Pipeline End-to-End

Berikut contoh ringkas pipeline klasifikasi tabular.

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

X = torch.randn(1000, 20)
y = torch.randint(0, 3, (1000,))

dataset = TensorDataset(X, y)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = nn.Sequential(
    nn.Linear(20, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 3)
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(10):
    model.train()
    total_loss = 0.0

    for inputs, targets in loader:
        inputs = inputs.to(device)
```

```
targets = targets.to(device)

logits = model(inputs)
loss = criterion(logits, targets)

optimizer.zero_grad()
loss.backward()
optimizer.step()

total_loss += loss.item() * inputs.size(0)

print(f"epoch={epoch}, loss={total_loss / len(dataset):.4f}")
```

33 Kesimpulan

PyTorch adalah framework deep learning yang menggabungkan fleksibilitas Python dengan efisiensi komputasi tensor. Konsep teknis utama PyTorch meliputi tensor, device, autograd, dynamic computation graph, modul `nn.Module`, training loop eksplisit, optimizer, dataset, dataloader, evaluasi, regularisasi, mixed precision, penyimpanan model, debugging, dan deployment.

Untuk menguasai PyTorch, pemahaman konseptual harus berjalan bersama praktik teknis. Seorang pengguna PyTorch perlu memahami bentuk tensor, alur gradien, perbedaan mode training dan evaluasi, sifat loss function, cara kerja optimizer, serta cara membuat pipeline data yang efisien. Dengan fondasi tersebut, PyTorch dapat digunakan untuk membangun berbagai model modern, mulai dari multilayer perceptron, CNN, RNN, LSTM, transformer, hingga sistem deep learning skala besar.